

[milan@datajoin.net](mailto:milan@datajoin.net)

<http://datajoin.net>

DataJoin

# Graph query for SQL database

[Document subtitle]

Milan Patel

5-2-2025

## Table of

## Contents

|                                                                     |                                     |
|---------------------------------------------------------------------|-------------------------------------|
| what is a graph database (graphdb1) .....                           | 2                                   |
| Typical sql schema design (sqldesign1) .....                        | 4                                   |
| entities nodes from various data (node1) .....                      | 6                                   |
| entities and relationship in sql tables (entrel1) .....             | 6                                   |
| Use dedicated node and edge table instead of views (nodeedge1)..... | <b>Error! Bookmark not defined.</b> |
| convert relationship matrix into key value pair (pymelt1) .....     | 8                                   |
| Graph traverse queries (graphquery1).....                           | 9                                   |
| Graph traverse queries (graphquery1).....                           | 10                                  |
| Find connection between two entities (nodes) (graphquery2) .....    | 12                                  |
| Pattern query (graphquery3) .....                                   | 13                                  |

## Overview

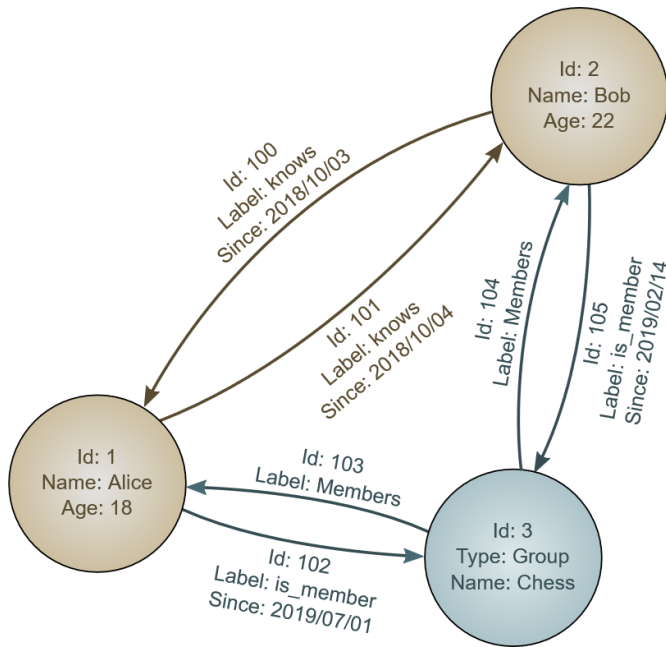
**Problem:** End users need to initiate data queries based on various parameters, such as time, region, customer type, or product type. These queries must retrieve data from a standard SQL table schema where foreign keys may not be explicitly defined, or the table design may be suboptimal. A standard SQL table join must be meticulously designed, developed, and tested for each possible combination of tables. Additionally, the process of transforming and storing data in a data warehouse is often associated with significant costs.

**Solution:** Design a database view that allows queries to be initiated using any entity type. Additionally, enable the exploration of entity types in any combination.

DataJoin.net provides in-depth education and consulting on graph queries for SQL tables.

Vocabulary: view is SQL database view, attribute or column is SQL table column.

## what is a graph database (graphdb1)



By Ole Mussmann - Own work, CC0,

<https://commons.wikimedia.org/w/index.php?curid=87002327>

A graph database is a database that is based on graph theory. It consists of a set of objects, which can be a node or an edge.

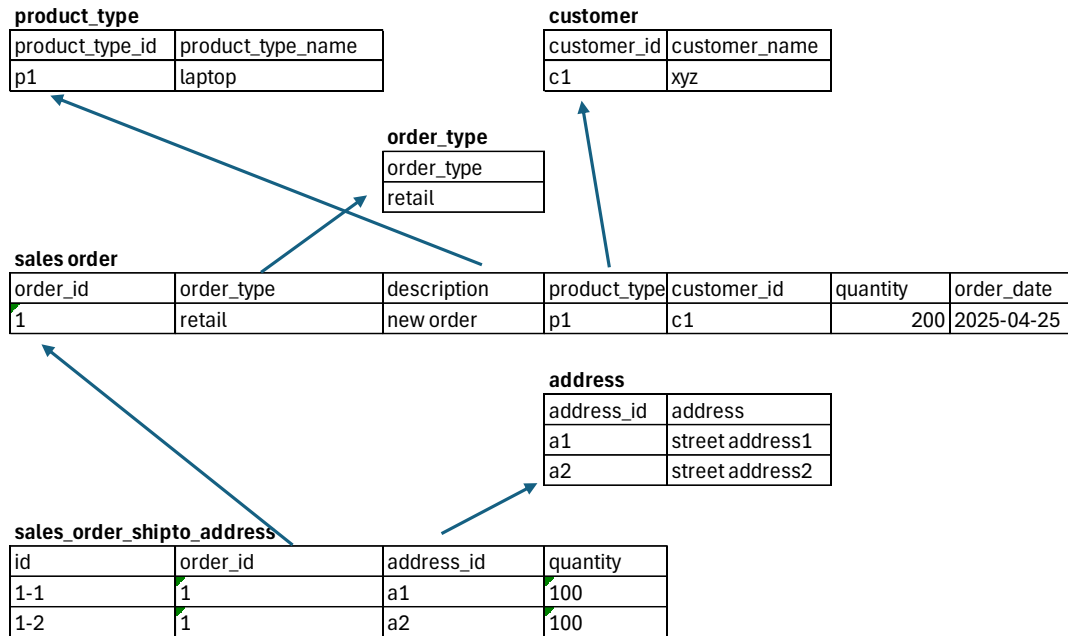
Nodes represent entities or instances such as people, businesses, accounts, or any other item to be tracked. They are roughly the equivalent of a record, relation, or row in a relational database, or a document in a document-store database.

Edges, also termed graphs or relationships, are the lines that connect nodes to other nodes; representing the relationship between them.

The edges can either be directed or undirected. In an undirected graph, an edge connecting two nodes has a single meaning. In a directed graph, the edges connecting two different nodes have different meanings, depending on their direction.

[https://en.wikipedia.org/wiki/Graph\\_database](https://en.wikipedia.org/wiki/Graph_database)

## Typical sql schema design (sqldesign1)



note1: customer and sales order tables are item(instance) tables

note2: product\_type and order\_type are category (class/type) tables

note3: sales\_order\_ship\_to\_address table is many to many relation between sales\_order and address tables

graph query objects.xlsx

It is necessary to know what nodes and relationships there in the schema are and which one is needed for Graph Query.

Nodes (entities) are Product type, order type, customer, sales order, address and sales order ship to address.

Relationships ae: sales order and product type, sales order and order type, sales order and customer, sales order and address (sale order ship to address)

<https://github.com/lerocha/chinook-database>

[https://github.com/LucasMcL/15-sql\\_queries\\_02-chinook](https://github.com/LucasMcL/15-sql_queries_02-chinook)

[https://github.com/bitmakerlabs/chinook/blob/master/doc/chinook\\_schema.png](https://github.com/bitmakerlabs/chinook/blob/master/doc/chinook_schema.png)

[https://github.com/xivSolutions/ChinookDb\\_Pg\\_Modified](https://github.com/xivSolutions/ChinookDb_Pg_Modified)

postgres specific schema and data

<https://github.com/cwoodruff/ChinookDatabase>



## Entities nodes from various tables(node1)

Common view for nodes needs at least 3 columns (entity\_id, entity\_type, description). If necessary, all other necessary common can be added such as organization\_id, or group\_id depending upon need. Typically, there is one view for each table and then combines all view of each table into one view that can show data from all tables.

|                         |                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------|
| <b>combine_nodeview</b> | CREATE VIEW combine_nodeview as                                                                          |
| sales_order_nodeview    | select order_id as entity_id,'sales_order' as entity_type,<br>description as entity_value UNION          |
| address_nodeview        | select address_id as entity_id,'address' as entity_type, address<br>as entity_value UNION                |
| order_type_nodeview     | select order_type as entity_id,'order_type' as entity_type,<br>order_type as entity_value UNION          |
| custom_nodeview         | select customer_id as entity_id,'customer' as entity_type,<br>customer_name as entity_value UNION        |
| product_type_nodeview   | select product_type_id as entity_id,'product_type' as<br>entity_type, product_type_name as entity_value; |

## Relationship from various tables (entrel1)

Common relationship (edge) view needs atleast 3 columns (from\_id, to\_id,predicate/relationship). If necessary, all other necessary common can be added such as organization\_id, or group\_id depending upon need. Typically there is one view for each table and then combines all view of each table into one view that can show data from all tables.

|                                      |                                                                                                                                                                                                                          |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>combine_relation_edgeview</b>     | CREATE VIEW combine_relation_edgeview as                                                                                                                                                                                 |
| sales_order_to_product_type_edgeview | SELECT order_id as from_id, product_type_id as to_id,'product_type' as<br>predicate FROM sales_order UNION<br>SELECT product_type_id as from_id, order_id as to_id,'product_type' as<br>predicate FROM sales_order UNION |
| sales_order_to_customer_edgeview     | SELECT order_id as from_id, customer_id as to_id,'customer' as predicate<br>FROM sales_order UNION<br>SELECT customer_id as from_id, order_id as to_id,'product_type' as predicate<br>FROM sales_order UNION             |

|                                    |                                                                                                                                                                                                            |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sales_order_to_order_type_edgeview | SELECT order_id as from_id, order_type as to_id,'order_type' as predicate<br>FROM sales_order UNION<br>SELECT order_type as from_id, order_id as to_id,'order_type' as predicate<br>FROM sales_order UNION |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                 |                                                                                                                                                                                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sales_order_to_address_edgeview | SELECT order_id as from_id, address_id as to_id,'order_address' as predicate<br>FROM sales_order_shipto_address UNION<br>SELECT address_id as from_id, order_id as to_id,'order_address' as predicate<br>FROM sales_order_shipto_address; |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Now you all data in nodes view and relationship view.

Data example of the combine\_nodeview and combine\_relation\_edgeview

## Example data of node view and relationship view

### combine\_nodeview

| entity_id | entity_type | entity_value    |
|-----------|-------------|-----------------|
| ord1      | retail      | new order       |
| p1        | laptop      | laptop          |
| c1        | customer    | xyz             |
| a1        | address     | street address1 |
| a2        | address     | street address2 |

### combine\_relation\_edgeview

| from_id | to_id  | predicate    |
|---------|--------|--------------|
| ord1    | p1     | product_type |
| ord1    | c1     | customer     |
| prd1    | a1     | address      |
| ord1    | a2     | address      |
| ord1    | retail | order_type   |

## Relationship is maintained in matrix format into key value pair (pymelt1)

It is natural to maintain a simple worksheet table show below to maintain relationship. The relationships are each pairs row and column. Now it is necessary to convert the matrix format into key value pair table.

**relation\_matrix\_algorithm\_to\_capability**

| row_id     | capability1 | capability2 | capability3 | capability4 |
|------------|-------------|-------------|-------------|-------------|
| algorithm1 |             | y           | y           |             |
| algorithm2 | y           |             |             |             |
| algorithm3 |             |             |             | y           |
| algorithm4 |             |             |             | y           |

use python program to convert the above matix into the following keyvalue pair

**relation\_keyvalue\_algorithm\_to\_capability**

| row_id     | object      | predicate |
|------------|-------------|-----------|
| algorithm1 | capability2 | y         |
| algorithm1 | capability3 | y         |
| algorithm2 | capability1 | y         |
| algorithm3 | capability4 | y         |
| algorithm4 | capability4 | y         |

Each matrix table must have corresponding key value pair table. This key value pair table will be created and overwritten by python algorithm. Then, create combine view that will pull data from all the tables. Each table can be overwritten using the python program.  
 SELECT row\_id as from\_id, object as to\_id, predicate as predicate FROM relation\_keyvalue\_algorithm\_to\_capability UNION  
 SELECT object as from\_id, row\_id as to\_id, predicate as predicate FROM relation\_keyvalue\_algorithm\_to\_capability UNION

## Graph traverse queries (graphquery1)

Graph traverse means: Find or pick the first entity, then, find which other entities have relationship with the first entity. This entity to entity search (or hopping) can continue as per need. This type of query is more suitable for category to category exploration since the counts of categories can be hundreds or thousands and not in millions. It is possible to construct a query so it would do the count of next entities first before it would actually try to retrieve large number of data as part of query. Also, it possible to avoid circular relationship (reference) that can duplicate data without any value. For example, A is related to B and then B is related A. A is related A itself. This type of relationship will keep showing results in each next hop without adding new information.

### combine\_nodeview

| entity_id | entity_type | entity_value    |
|-----------|-------------|-----------------|
| ord1      | retail      | new order       |
| p1        | laptop      | laptop          |
| c1        | customer    | xyz             |
| a1        | address     | street address1 |
| a2        | address     | street address2 |

### combine\_relation\_edgeview

| from_id | to_id  | predicate    |
|---------|--------|--------------|
| ord1    | p1     | product_type |
| ord1    | c1     | customer     |
| prd1    | a1     | address      |
| ord1    | a2     | address      |
| ord1    | retail | order_type   |

SELECT

```

n1.entity_id from_entity_id
,n1.entity_type from_entity_type
,n1.attr_char from_entity_value
,reln1.predicate Relationship
,n2.entity_id to_entity_id
,n2.entity_type to_entity_type

```



```
,n2.entity_value to_entity_value
```

```
FROM
combine_nodeview n1
,combine_edgeview reln1
,combine_nodeview n2
WHERE
reln1.from_id = n1.entity_id
And reln1.to_id = n2.entity_id
and n1.row_id='NEXT ENTITY ID'
order by from_entity_type,from_entity_value,to_entity_type,to_entity_value;"
```

## Graph traverse queries filters

Views can have the following filter field. Dedicated table may have the following filters

Organization

Graph group

Active flag

Connector type (category or item)

Counts as filter

Organization filter (if org\_id is used in node and edge)

```
sSQL_nexthop_last = ""
sSQL_nexthop_last = sSQL_nexthop_last + " and n1.org_id in ('" + hdn_dataorg.Text + "')"
sSQL_nexthop_last = sSQL_nexthop_last + " and n2.org_id in ('" + hdn_dataorg.Text + "')"
sSQL_nexthop_last = sSQL_nexthop_last + " and reln1.org_id in ('" + hdn_dataorg.Text + "')
```

active or inactive filter (if active\_flag is used or applied during node and edge view)

```
If hdn_current_role.Text <> "ttadmin" Or hdn_current_role.Text = "" Then
sSQL_nexthop_last = sSQL_nexthop_last + " and n1.active_flag='Y' and n2.active_flag='Y' and reln1.active_flag='Y' "
End If
```

Connector type are: category, item, etc. (if item\_cat field is used)

```
sSQL_nexthop_last = sSQL_nexthop_last + " and reln1.connector_type=conn1.rowid and conn1.connector_type='" +
ddl_connectortype2.SelectedValue + "'"
End If
```



```

If DDL_GoodVsBad.SelectedValue <> "All" And hdn_searchblock.Text <> "Simple Search" Then
    sSQL_nexthop_last = sSQL_nexthop_last + " and reln1.good_bad = '" + DDL_GoodVsBad.SelectedValue + "'"
End If

```

If graph relation has group attribute

```

If DDL_TopicGroup.SelectedValue <> "All" Then
    sSQL_nexthop_last = sSQL_nexthop_last + " and reln1.graph_group= '" + DDL_TopicGroup.SelectedValue + "'"
End If

```

Graph traverse queries circular reference avoidance

## Find connection between two entities (nodes) (graphquery2)

Typical example such query is person to person, person to phone number, person to address scenarios. If you have names of two persons and you want to find if they are related to each other through other people or addresses. For graph queries for sql, it is necessary to construct separate query for each hop. Thus, a one hop query would produce result if two entities are connected directly (one hop). For example, Person A and Person B are brothers. A two hop query would produce results if two entities are related to each other through other entities. For example, person A lived at Address X and then Address X is also related to Person B (Person B lived at Address X).

```
select ' 1- ' + IIF(reln1.connectordefinition is NULL,n1.attr_char+' is related to:',reln1.connectordefinition) + ' >>>2- ' +
IIF(reln2.connectordefinition is NULL,n1.attr_char+' is related to:',reln2.connectordefinition) Relation2 FROM stg_nodetable n1,
stg_MODEL_ATTRIBUTE_RELN reln1, stg_nodetable n2, stg_MODEL_ATTRIBUTE_RELN reln2, stg_nodetable n3,st_connector_type
conn1,st_connector_type conn2 WHERE MATCH (n1-(reln1)->n2-(reln2)->n3) and reln1.attribute_reln_key<>reln2.attribute_reln_key and
n1.active_flag='Y' and n2.active_flag='Y' and n3.active_flag='Y' and reln1.active_flag='Y' and reln2.active_flag='Y'"
sSQL123 = sSQL123 + " and n1.org_id in ('" + hdn_dataorg.Text + "')"
sSQL123 = sSQL123 + " and n2.org_id in ('" + hdn_dataorg.Text + "')"
sSQL123 = sSQL123 + " and n3.org_id in ('" + hdn_dataorg.Text + "')"
sSQL123 = sSQL123 + " and reln1.org_id in ('" + hdn_dataorg.Text + "')"
sSQL123 = sSQL123 + " and reln2.org_id in ('" + hdn_dataorg.Text + "')"
sSQL123 = sSQL123 + " and n1.row_id='" + hdn_src_rowid.Text + "' and n3.row_id='" + hdn_dest_rowid.Text + "'"
```

### Two hop query

SELECT

n1.entity\_id from\_entity\_id,n1.entity\_type from\_entity\_type,n1.attr\_char from\_entity\_value  
,reln1.predicate Relationship1,reln2.predicate Relationship2  
,n3.entity\_id to\_entity\_id,n3.entity\_type to\_entity\_type,n3.entity\_value to\_entity\_value

FROM

combine\_nodeview n1  
,combine\_edgeview reln1  
,combine\_nodeview n2  
,combine\_edgeview reln2  
,combine\_nodeview n3

WHERE

n1.entity\_id = reln1.from\_id  
and reln1.to\_id = n2.entity\_id



```

n2.entity_id= reln2.from_id
reln2.to_id = n3.entity_id
and reln1.to_id = n3.entity_id
and reln2.to_id<>n1.entity_id
--to unselect all the from entities of the previous relation. Thus, circular reference is avoided
and n1.row_id='starting entity_id'
and n3.row_id='ending entity_id'

```

## Pattern query (graphquery3)

Pattern is defined as: Person A visits Coffee shop B. Another pattern can be Person A calls Person B. In these example, it would be necessary to find how many time this a pattern is repeated over time.

Pattern is relationship. A pattern (relationship) can be associated with multiple starting entity. Thus, only the relationship associated with starting entity will be selected.

Dedicated relationship table is needed for pattern to starting entity association

## Pattern design for graph query

<ul>

<li>Prerequisite Step 1 Create Pattern Node - A specific pattern is a node where entity\_id\_linked is rowid of first node for the pattern, enter Pattern in Entity\_type and Root\_entity. This will show up in the drop-down list as available pattern to choose. </li>

<li>Prerequisite Step 2 Relationships are already in the system for its own reason</li>

<li>Pattern Creation steps: </li>

<li>Step 1 Pick the pattern from the drop-down list and select. This GUID show in Pattern RowId and it will be used to associate with multiple relationship GUIDs. </li>

<li>Step 2: Search Relationship to be associated with Pattern Node. </li> (this DDL is from the Edit Relation search box)

<li>Step 3: Connection to Insert: Pick the relationship from the drop-down list and Click Insert to add the relation</li>

</ul>  
<br />

| row_id                               | attribute_reln_key                   |
|--------------------------------------|--------------------------------------|
| 058F3477-23EB-48EF-8046-FD998F2E8696 | B0639C2A-6D0C-58E4-1BD2-EB20F38A36F0 |
| 058F3477-23EB-48EF-8046-FD998F2E8696 | 7cfd84e-70f6-4911-be80-52d6c65dd7bc  |
| 058F3477-23EB-48EF-8046-FD998F2E8696 | 8b10b056-0669-43a1-ac72-b9af27feef09 |

FROM [dbo].[ST\_PATTERN\_RELN]

One relationship id can be used with multiple pattern also using the above table

Pattern should be one direction : Left to Right

The following is simplistic pattern with one relationship within a HOP

### Connection Sequence

Some Politicians may exploit Fear

Some Politicians may exploit Fear >>> Fear leads to Hate

Some Politicians may exploit Fear >>> Fear leads to Hate >>> Hate leads to War/Civil War/Unrest/Genocide

## Most important entity query (graphquery3)

It may be necessary to find out who is the person most people calls or sends email. Another example would be which is most visited place by type of people.